1998. GCD Sort of an Array

leetcode.com/problems/gcd-sort-of-an-array/description/

Problem List | Submit | Premium

Description | Editorial | Solutions | Submissions

1998. GCD Sort of an Array

Solved

Hard | Topics | Companies | Hint

You are given an integer array `nums`, and you can perform the following operation **any** number of times on `nums`:

- Swap the positions of two elements `nums[i]` and `nums[j]` if `gcd(nums[i], nums[j]) > 1` where `gcd(nums[i], nums[j])` is the **greatest common divisor** of `nums[i]` and `nums[j]`.

Return `true` *if it is possible to sort* `nums` *in* ***non-decreasing*** *order using the above swap method, or* `false` *otherwise.*

Example 1:

```
Input: nums = [7,21,3]
Output: true
Explanation: We can sort [7,21,3] by performing the following operations:
- Swap 7 and 21 because gcd(7,21) = 7. nums = [21,7,3]
- Swap 21 and 3 because gcd(21,3) = 3. nums = [3,7,21]
```

Example 2:

```
Input: nums = [5,2,6,2]
Output: false
Explanation: It is impossible to sort the array because 5 cannot be swapped with any other element.
```

Code

C | Auto

```c
#include <stdlib.h>

#define MAX 100001

int parent[MAX];
int size[MAX];
int spf[MAX];

// Find with path compression
int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

// Union by size
void unionSet(int a, int b) {
    int rootA = find(a);
    int rootB = find(b);
    if (rootA != rootB) {
        if (size[rootA] < size[rootB]) {
            parent[rootA] = rootB;
            size[rootB] += size[rootA];
        } else {
            parent[rootB] = rootA;
            size[rootA] += size[rootB];
        }
    }
```

Saved | Ln 79, Col 21

SHIRISHA B R

Access all features with our Premium subscription!

My Lists | Notebook | Progress | Points

Try New Features | Orders | My Playgrounds | Settings | Appearance | Sign Out

```c
}

// Build smallest prime factor array
void buildSPF() {
    for (int i = 0; i < MAX; i++)
        spf[i] = i;

    for (int i = 2; i * i < MAX; i++) {
        if (spf[i] == i) {
            for (int j = i * i; j < MAX; j += i) {
                if (spf[j] == j)
                    spf[j] = i;
            }
        }
    }
}

// Compare function for qsort
int compare(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

bool gcdSort(int* nums, int numsSize) {
    buildSPF();

    for (int i = 0; i < MAX; i++) {
        parent[i] = i;
        size[i] = 1;
```

```c
    }

    // Union numbers with their prime factors
    for (int i = 0; i < numsSize; i++) {
        int x = nums[i];
        while (x > 1) {
            int prime = spf[x];
            unionSet(nums[i], prime);
            while (x % prime == 0)
                x /= prime;
        }
    }

    int* sorted = (int*)malloc(numsSize * sizeof(int));
    for (int i = 0; i < numsSize; i++)
        sorted[i] = nums[i];

    qsort(sorted, numsSize, sizeof(int), compare);

    for (int i = 0; i < numsSize; i++) {
        if (find(nums[i]) != find(sorted[i])) {
            free(sorted);
            return false;
        }
    }

    free(sorted);
    return true;
}
```

Example 3:

```
Input: nums = [10,5,9,3,15]
Output: true
We can sort [10,5,9,3,15] by performing the following operations:
- Swap 10 and 15 because gcd(10,15) = 5. nums = [15,5,9,3,10]
- Swap 15 and 3 because gcd(15,3) = 3. nums = [3,5,9,15,10]
- Swap 10 and 15 because gcd(10,15) = 5. nums = [3,5,9,10,15]
```

Constraints:

- `1 <= nums.length <= 3 * 10^4`
- `2 <= nums[i] <= 10^5`

Seen this question in a real interview before? 1/5

Testcase | Test Result

Accepted Runtime: 2 ms

Case 1 Case 2 Case 3

Input

nums =
[7,21,3]

Output

true

Expected

true

Testcase | Test Result

Accepted Runtime: 2 ms

Case 1 Case 2 Case 3

Input

nums =

[5,2,6,2]

Output

false

Expected

false

Testcase | Test Result

Accepted Runtime: 2 ms

Case 1 Case 2 Case 3

Input

nums =

[10,5,9,3,15]

Output

true

Expected

true

3312. Sorted GCD Pair Queries

Problem List | Submit | Premium

Description | Editorial | Solutions | Submissions

3312. Sorted GCD Pair Queries

Solved

Hard | Topics | Companies | Hint

You are given an integer array `nums` of length `n` and an integer array `queries`.

Let `gcdPairs` denote an array obtained by calculating the GCD of all possible pairs `(nums[i], nums[j])`, where `0 <= i < j < n`, and then sorting these values in **ascending** order.

For each query `queries[i]`, you need to find the element at index `queries[i]` in `gcdPairs`.

Return an integer array `answer`, where `answer[i]` is the value at `gcdPairs[queries[i]]` for each query.

The term `gcd(a, b)` denotes the **greatest common divisor** of `a` and `b`.

Example 1:

> **Input:** `nums = [2,3,4], queries = [0,2,2]`
>
> **Output:** `[1,2,2]`

Code | Accepted

C | Auto

```c
#include <stdlib.h>

int* gcdValues(int* nums, int numsSize, long long* queries, int queriesSize, int* returnSize){
    int maxV = 50000;
    int* freq = (int*)calloc(maxV + 1, sizeof(int));
    for(int i = 0; i < numsSize; i++) freq[nums[i]]++;

    long long* count = (long long*)calloc(maxV + 1, sizeof(long long));

    // Count pairs divisible by each GCD
    for(int g = maxV; g >= 1; g--){
        long long total = 0;
        for(int multiple = g; multiple <= maxV; multiple += g){
            total += freq[multiple];
        }
        count[g] = total * (total - 1) / 2; // all pairs divisible by g

        // Inclusion-exclusion: remove multiples of larger divisors
        for(int multiple = 2*g; multiple <= maxV; multiple += g){
            count[g] -= count[multiple];
        }
    }
```

Problem List | Submit | Premium

Description | Editorial | Solutions | Submissions

Explanation:

> `gcdPairs = [gcd(nums[0], nums[1]), gcd(nums[0], nums[2]), gcd(nums[1], nums[2])] = [1, 2, 1]`.
>
> After sorting in ascending order, `gcdPairs = [1, 1, 2]`.
>
> So, the answer is `[gcdPairs[queries[0]], gcdPairs[queries[1]], gcdPairs[queries[2]]] = [1, 2, 2]`.

Example 2:

> **Input:** `nums = [4,4,2,1], queries = [5,3,1,0]`
>
> **Output:** `[4,2,1,1]`
>
> **Explanation:**
>
> `gcdPairs` sorted in ascending order is `[1, 1, 1, 2, 2, 4]`.

Example 3:

> **Input:** `nums = [2,2], queries = [0,0]`
>
> **Output:** `[2,2]`
>
> **Explanation:**
>
> `gcdPairs = [2]`.

Code | Accepted

C | Auto

```c

    // Build prefix sum
    long long* prefix = (long long*)malloc((maxV+2) * sizeof(long long));
    prefix[0] = count[0];
    for(int i = 1; i <= maxV; i++) prefix[i] = prefix[i-1] + count[i];

    // Answer queries using binary search on prefix sums
    int* ans = (int*)malloc(queriesSize * sizeof(int));
    for(int i = 0; i < queriesSize; i++){
        long long q = queries[i]+1; // 0-based index -> 1-based
        int l = 1, r = maxV, res = 0;
        while(l <= r){
            int m = (l+r)/2;
            if(prefix[m] >= q){
                res = m;
                r = m-1;
            }else l = m+1;
        }
        ans[i] = res;
    }

    free(freq);
    free(count);
    free(prefix);
    *returnSize = queriesSize;
    return ans;
}
```

Example 3:

> **Input:** `nums = [2,2], queries = [0,0]`
>
> **Output:** `[2,2]`
>
> **Explanation:**
>
> `gcdPairs = [2]`.

Constraints:

- `2 <= n == nums.length <= 10^5`
- `1 <= nums[i] <= 5 * 10^4`
- `1 <= queries.length <= 10^5`
- `0 <= queries[i] < n * (n - 1) / 2`

Accepted Runtime: 11 ms

Case 1 | Case 2 | Case 3

Input

nums =
`[2,3,4]`

queries =
`[0,2,2]`

Output

`[1,2,2]`

Expected

`[1,2,2]`

Testcase | Test Result

Accepted Runtime: 11 ms

Case 1 Case 2 Case 3

Input

nums =

[4,4,2,1]

queries =

[5,3,1,0]

Output

[4,2,1,1]

Expected

[4,2,1,1]

Testcase | Test Result

Accepted Runtime: 11 ms

Case 1 Case 2 Case 3

Input

nums =

[2,2]

queries =

[0,0]

Output

[2,2]

Expected

[2,2]

3658. GCD of Odd and Even Sums

Problem List | Submit | Premium

Description | Editorial | Solutions | Submissions

3658. GCD of Odd and Even Sums

Solved

Easy | Topics | Companies | Hint

You are given an integer `n`. Your task is to compute the **GCD** (greatest common divisor) of two values:

- `sumOdd`: the sum of the smallest `n` positive odd numbers.
- `sumEven`: the sum of the smallest `n` positive even numbers.

Return the GCD of `sumOdd` and `sumEven`.

Example 1:

Input: n = 4

Output: 4

Explanation:

- Sum of the first 4 odd numbers `sumOdd = 1 + 3 + 5 + 7 = 16`
- Sum of the first 4 even numbers `sumEven = 2 + 4 + 6 + 8 = 20`

Hence, `GCD(sumOdd, sumEven) = GCD(16, 20) = 4`.

Code | Accepted

C | Auto

```c
int gcdOfOddEvenSums(int n) {
    // Sum of first n odd numbers = n^2
    // Sum of first n even numbers = n(n+1)
    // gcd(n^2, n(n+1)) = n

    return n;
}
```

SHIRISHA B R

Access all features with our Premium subscription!

My Lists | Notebook | Progress | Points

Try New Features

Orders

My Playgrounds

Settings

Appearance

Sign Out

Description | Editorial | Solutions | Submissions

Example 2:

Input: n = 5

Output: 5

Explanation:

- Sum of the first 5 odd numbers `sumOdd = 1 + 3 + 5 + 7 + 9 = 25`
- Sum of the first 5 even numbers `sumEven = 2 + 4 + 6 + 8 + 10 = 30`

Hence, `GCD(sumOdd, sumEven) = GCD(25, 30) = 5`.

Constraints:

- `1 <= n <= 1000`

Code | Accepted

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

n =
4

Output

4

Expected

4

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

n =
5

Output

5

Expected

5

2447. Number of Subarrays With GCD Equal to K

Problem List

Submit

Premium

Description | Editorial | Solutions | Submissions

2447. Number of Subarrays With GCD Equal to K

Solved

Medium | Topics | Companies | Hint

Given an integer array `nums` and an integer `k`, return *the number of **subarrays** of* `nums` *where the greatest common divisor of the subarray's elements is* `k`.

A **subarray** is a contiguous non-empty sequence of elements within an array.

The **greatest common divisor of an array** is the largest integer that evenly divides all the array elements.

Example 1:

```
Input: nums = [9,3,1,2,6,3], k = 3
Output: 4
Explanation: The subarrays of nums where 3 is the greatest
common divisor of all the subarray's elements are:
- [9,3,1,2,6,3]
- [9,3,1,2,6,3]
- [9,3,1,2,6,3]
- [9,3,1,2,6,3]
```

Code | Accepted

C | Auto

```c
long long gcd(long long a, long long b) {
    while (b) {
        long long t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int subarrayGCD(int* nums, int numsSize, int k) {
    int count = 0;

    for (int i = 0; i < numsSize; i++) {
        long long currentGCD = 0;

        for (int j = i; j < numsSize; j++) {
            currentGCD = gcd(currentGCD, nums[j]);

            // If GCD becomes smaller than k, no need to continue
            if (currentGCD < k)
                break;

            if (currentGCD == k)
                count++;
        }
    }

    return count;
```

SHIRISHA B R

Access all features with our Premium subscription!

My Lists | Notebook | Progress | Points

Try New Features

Orders

My Playgrounds

Settings

Appearance

Sign Out

Example 1:

```
Input: nums = [9,3,1,2,6,3], k = 3
Output: 4
Explanation: The subarrays of nums where 3 is the greatest common divisor of
all the subarray's elements are:
- [9,3,1,2,6,3]
- [9,3,1,2,6,3]
- [9,3,1,2,6,3]
- [9,3,1,2,6,3]
```

Example 2:

```
Input: nums = [4], k = 7
Output: 0
Explanation: There are no subarrays of nums where 7 is the greatest common
divisor of all the subarray's elements.
```

Constraints:

- `1 <= nums.length <= 1000`
- `1 <= nums[i], k <= 10^9`

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

nums =
[9,3,1,2,6,3]

k =
3

Output

4

Expected

4

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =

[4]

k =

7

Output

0

Expected

0